

Binary Tree Traversals

A traversal visits every node exactly once. The order in which we visit nodes defines the traversal type. Let's use this balanced BST as our example:

```
{type: "tree", nodes: "4:2:6 2:1:3 6:5:7 1:-:- 3:-:- 5:-:- 7:-:-"}
```

Inorder (Left $\rightarrow$ Root $\rightarrow$ Right)

Recurse into the left subtree, visit the current node, then recurse into the right subtree. For a BST, this yields sorted order!

Result: 1 2 3 4 5 6 7

Preorder (Root $\rightarrow$ Left $\rightarrow$ Right)

Visit the current node first, then recurse left, then right. Useful for copying or serializing a tree.

Result: 4 2 1 3 6 5 7

Postorder (Left $\rightarrow$ Right $\rightarrow$ Root)

Recurse left, recurse right, then visit the current node last. Useful for deleting trees or evaluating expression trees.

Result: 1 3 2 5 7 6 4

Level-Order / BFS (Top $\rightarrow$ - Bottom, Left $\rightarrow$ Right)

Visit nodes level by level using a queue. Pop a node, enqueue its children, repeat.

Result:

4	2	6	1	3	5	7
---	---	---	---	---	---	---

Comparison Table

	Order	Result
In	middle	5
Pre	first	3 6
Post	last	5 7
Level	row by row	4 2 6 1 3 5 7

Mnemonic: the name tells you when the root is visited!

Pre = root first, In = root in middle, Post = root last. Level = row by row.
--